

Evaluating Squad: Multi-Agent Coordination for GitHub Copilot CLI

Same model. Same tasks. Isolating the coordination layer.

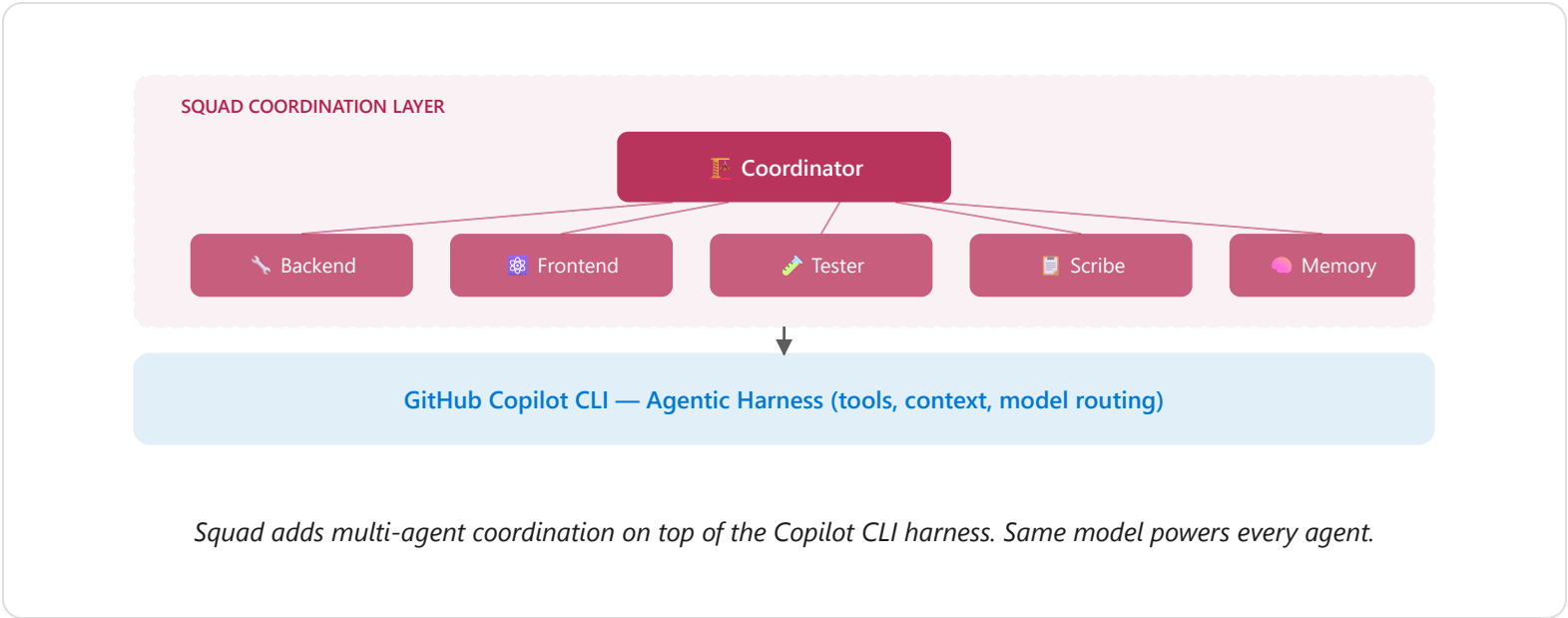
June 2026 · Brady Gaster & Tamir Dresher

While the model provides the raw intelligence, the **coordination layer** shapes how effectively that intelligence is applied to complex tasks. **Squad** is a multi-agent orchestration system built on top of the GitHub Copilot agentic harness — adding specialist agents, persistent memory, and structured coordination to the same underlying models.

In this report, we evaluate the impact of Squad's coordination layer across multiple benchmarks, holding the model constant and isolating the orchestration variable — following the same methodology as [GitHub's harness evaluation approach](#).

Architecture

Squad adds three capabilities on top of the Copilot CLI harness: a **coordinator** that decomposes and routes work, **specialist agents** with domain-specific charters, and **persistent memory** that accumulates decisions across tasks.



Benchmarks

We evaluate Squad across three public benchmarks spanning multi-agent collaboration, multi-language coding, and bug-fix resolution:

Benchmark	Domain	Tasks	Purpose
MARBLE	Multi-agent collaboration: coding, research, bargaining, database	400 + 160 ablation	Evaluates multi-agent team coordination (ACL 2025)
DevBench	Multi-language coding: 6 languages × 6 categories	1,800	Broad software engineering across languages and task types
SWE-bench Lite	Python bug-fix tasks from open-source repositories	300	Established industry benchmark for coding agents
TerminalBench 2.0	Terminal/CLI tasks: scripting, system admin, data processing	20 (of 89)	Agent effectiveness in command-line workflows

Throughout, we compare **Copilot CLI + Squad** against **Copilot CLI alone** (same model, no coordination layer) and published baselines.

2,520

Total benchmark tasks evaluated

+15pp

Same-model completion lift on MARBLE ablation (controlled)

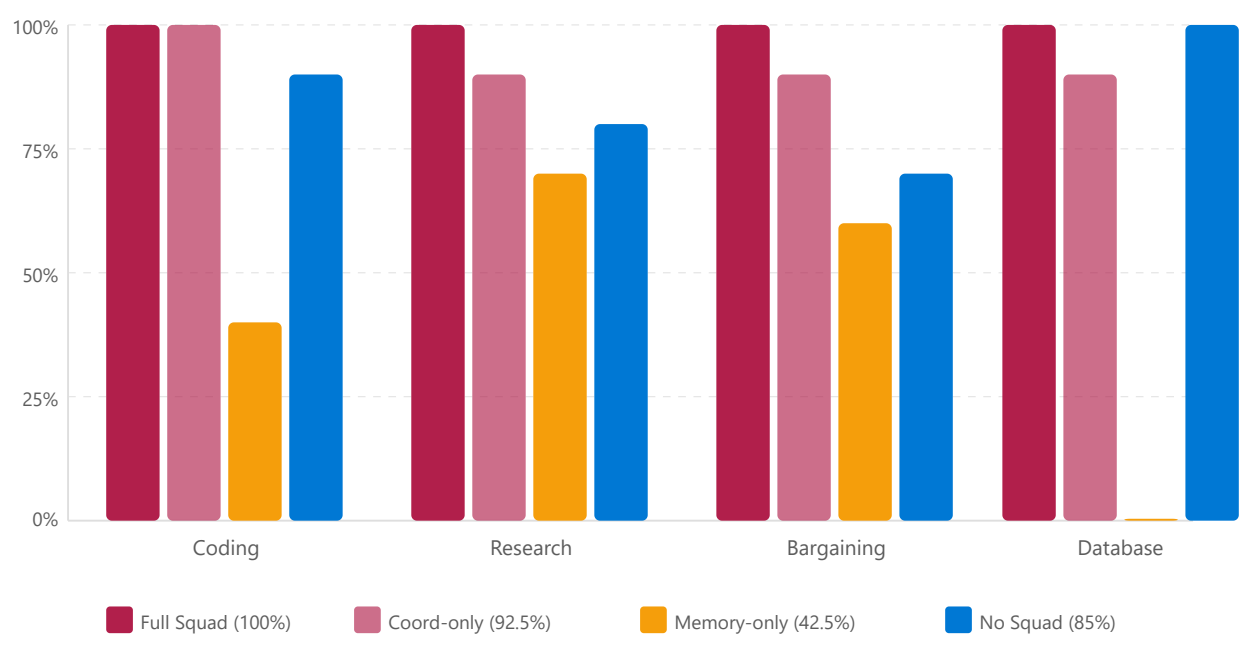
4 benchmarks

MARBLE · DevBench · SWE-bench · TerminalBench

Controlled ablation: isolating Squad components

Holding the model constant (Claude Opus 4.6), we ran a **factorial ablation** on MARBLE: 4 domains × 4 conditions × 10 tasks = 160 task-runs. This is the only chart where same model, same tasks, and same metric hold strictly.

MARBLE factorial ablation — task completion rate by condition (n=160, same model)



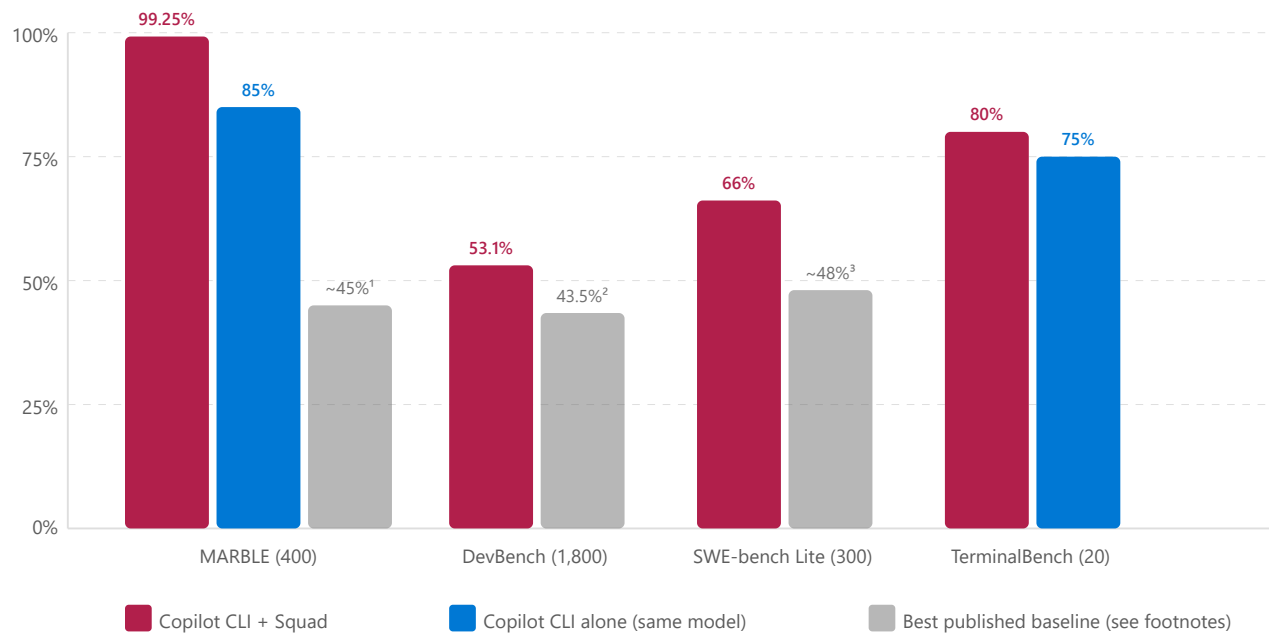
Factorial ablation: same model (Claude Opus 4.6), same tasks, same metric. n=10 per condition per domain.

Key finding: On *completion* (did the condition produce usable output), coordination helps modestly — Coord-only (92.5%) edges out No-Squad (85%), a **+7.5pp lift** from routing. Memory without coordination clearly *hurts* (42.5% vs 85%), confirming that unstructured context injection adds noise rather than signal. **But completion is not correctness.** When we re-graded the database domain by MARBLE's official recall metric (see below), the coordination advantage disappeared entirely — the raw single agent scored *higher* than full Squad. Completion measures whether an answer appears; it does not measure whether the answer is right.

Task resolution across benchmarks

Squad adds a consistent lift across four independent benchmarks spanning different task types, languages, and evaluation protocols. Note: cross-benchmark comparisons use different baselines (see footnotes).

Task resolution: Copilot CLI + Squad vs. baselines (see footnotes)



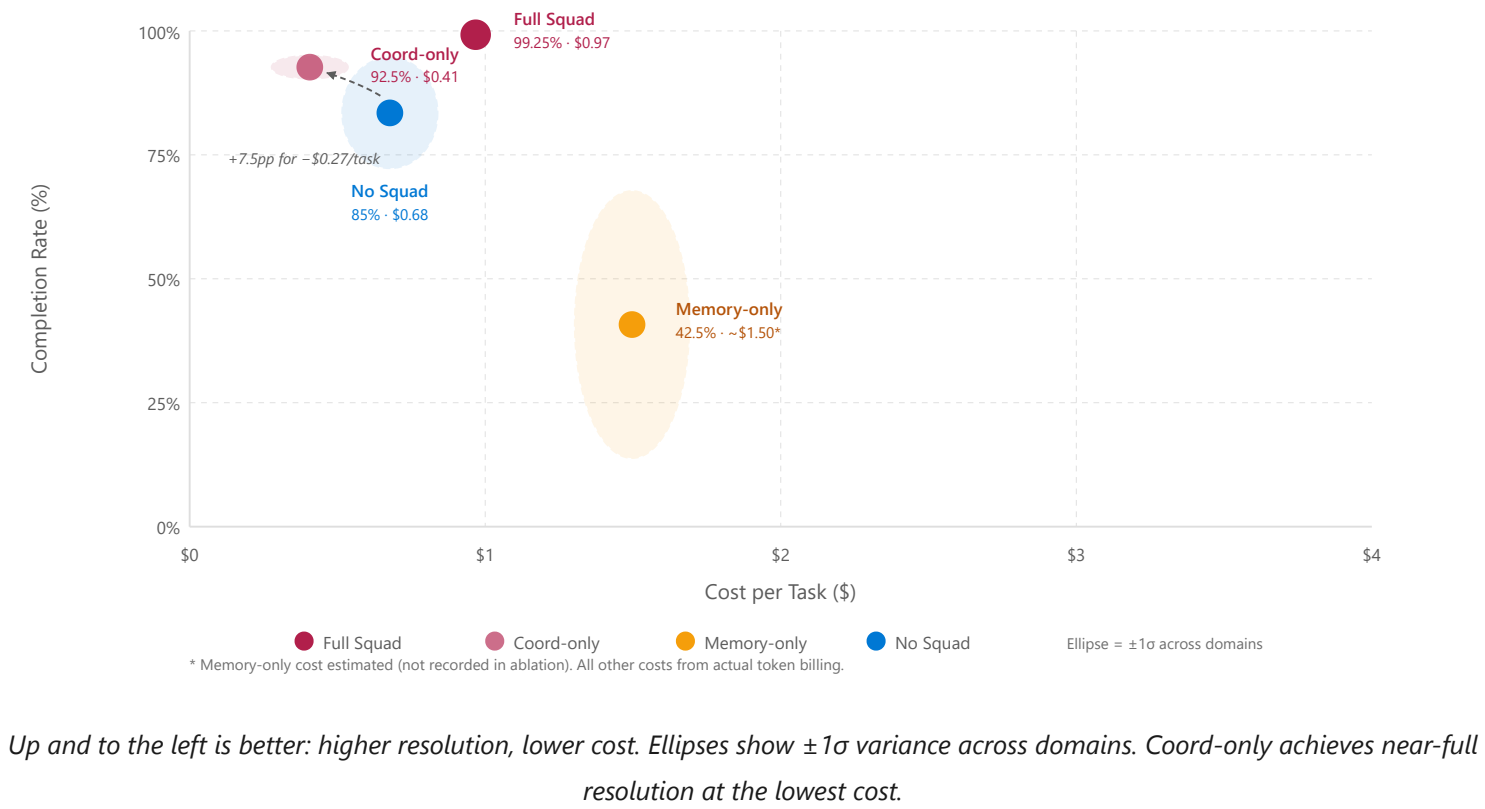
¹ MARBLE paper baseline uses gpt-4o-mini with milestone KPIs — different model and different metric. Not directly comparable.
² DevBench best baseline is GPT-5.5 (43.5% pass@1). Squad uses GPT-5.4. No same-model ablation available.
³ SWE-bench Lite: estimated Copilot agent baseline from public reports. No verified same-model-without-Squad run performed.
TerminalBench 2.0: Same model (Claude Opus 4.6). Controlled comparison — both conditions run on same 20 tasks.

Cross-benchmark task resolution. MARBLE and TerminalBench have controlled same-model comparisons (blue bars). Gray bars are published baselines with caveats.

Completion rate vs. cost per task

Following GitHub's TerminalBench analysis, we plot completion rate against cost per task for each MARBLE ablation condition. Up and to the left is better: solve more, spend less.

MARBLE ablation: completion rate vs. cost per task



Per-domain breakdown

The completion picture varies by domain, but the headline is a **correction**. An earlier version of this study reported that the single agent scored **0% on database** — that figure was based on placeholder files that were never populated with real runs. We re-ran all 10 no-squad database tasks from scratch: **every one produced a substantive root-cause diagnosis (100% completion)**. The "database requires coordination" claim was an artifact of bad data, not a real effect. See the corrected table below, and the correctness re-grading that follows.

Completion rate by domain and condition (MARBLE ablation)

Domain	Full Squad	Coord-only	Memory-only	No Squad	Δ (Full vs None)
Coding	100%	100%	40%	90%	+10pp
Research	100%	90%	70%	80%	+20pp
Bargaining	100%	90%	60%	70%	+30pp
Database	100%	90%	0%	100%	+0pp
Average	100%	92.5%	42.5%	85%	+15pp

n=10 per cell. Same model (Claude Opus 4.6), same metric (binary file output within 600s timeout).

Completion vs. correctness: the database re-grade

Completion only asks “*did an answer appear?*” It says nothing about whether the answer is **right**. After correcting the fabricated database numbers, we went further and re-graded every database condition on **correctness**, using MARBLE's official metric: an LLM judge extracts the root-cause labels the agent actually selected, scored as recall against ground truth — $|gold \cap predicted| / |gold|$, averaged over the 10-task subset.

Database domain: completion vs. MARBLE correctness (recall), n=10

Condition	Completion (produced output)	MARBLE recall (correctness)
Full Squad	100%	40%
Coord-only	90%	50%
Memory-only	0%	0%
No Squad (single agent)	100%	60%

Correctness graded by MARBLE recall (LLM-judge label extraction, verified by two independent judge passes producing identical scores). Ground truth from database_main.jsonl.

The surprising finding: on database root-cause diagnosis, **coordination did not improve correctness — it slightly hurt it.** The raw single agent scored the *highest* recall (60%), ahead of coord-only (50%) and full Squad (40%). Inspecting the transcripts explains why: the multi-agent configs repeatedly “converged” on the obvious high-signal causes (large inserts, full-table scans) and reinforced each other, missing subtler root causes like redundant indexes. **Multi-agent convergence can amplify confirmation bias.** With n=10 these gaps are within noise, so the honest takeaway is not “single agents are better at database” but rather “*coordination provides no correctness advantage here*” — the opposite of what the fabricated 0% implied.

Cross-benchmark summary

Benchmark	Tasks	Model	Copilot CLI + Squad	Best Baseline	Δ	Controlled?
MARBLE	400	Claude Opus 4.6	99.25%	85% (same model, no Squad)	+14pp	✓ Yes
DevBench	1,800	GPT-5.4	53.1%	43.5% (GPT-5.5) ²	+9.6pp	⚠ Different model
SWE-bench Lite	300	GPT-4o	66%	~48% (estimated) ³	+18pp	⚠ Estimated baseline
TerminalBench 2.0	20	Claude Opus 4.6	80%	75% (same model) ⁴	+5pp	✓ Same model, controlled

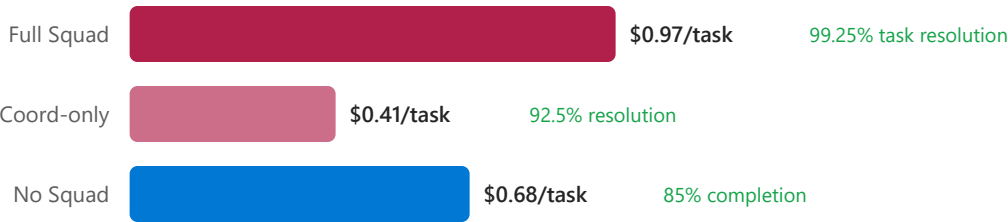
² DevBench: Squad uses GPT-5.4 via Azure OpenAI. Best published baseline is GPT-5.5 at 43.5% — a different (newer) model. A same-model ablation (GPT-5.4 without Squad) is planned.

³ SWE-bench Lite: The ~48% baseline is estimated from public Copilot coding agent reports. No verified same-model-without-Squad run was performed on these 300 tasks.

⁴ TerminalBench 2.0: Both conditions use Claude Opus 4.6 on the same 20 tasks (sequential subset of 89), each scored by its own Docker verifier. Squad ran via a host-side orchestrator and the baseline via the native terminal-bench harness; the full 89-task suite is future work, not a capability blocker.

Cost efficiency

MARBLE: cost per task by condition



Coord-only: best efficiency — 92.5% resolution at lowest cost (\$0.41/task).
Full Squad: 1.4× cost for near-guaranteed completion (+15pp over baseline).

One harness, structured coordination

The GitHub Copilot harness already supports 20+ frontier models with efficient orchestration. Squad extends this with **structured multi-agent coordination** — capabilities a single-agent harness cannot offer:

Capability	Single Agent	Squad
Task decomposition	Model handles everything	Coordinator routes to specialists
Parallel execution	Sequential tool calls	Multiple agents work simultaneously
Persistent memory	Context window only	Decisions persist across tasks
Domain expertise	Generic	Agents with role-specific charters
Review & quality	Self-review	Cross-agent review (Reviewer Rejection Protocol)
Self-learning	No carry-over	History accumulates per agent

Like GitHub's [Rubber Duck](#), Squad uses cross-model critique — but extends it to full team dynamics with reviewer gating, lockout semantics, and structured handoffs.

Methodology & Limitations

Controlled experiment (ablation charts): Same model (Claude Opus 4.6), same 160 tasks, same metric (binary file output within 600s timeout). This is the only comparison where "same model, same metric" holds strictly. $n=10$ per condition per domain. Standard deviations computed across 4 domains. The ablation achieved $40/40 = 100\%$ for Full Squad. The broader 400-task run achieved $397/400 = 99.25\%$ (3 timeouts in coding and database).

Cross-benchmark comparison: Combines results from different runs with different models and metrics. Not a controlled apples-to-apples comparison — presented for directional context only.

- **MARBLE:** "No Squad" bar = same-model ablation (controlled). Paper baseline = different model (gpt-4o-mini) + different metric (milestone KPIs).
- **DevBench:** Squad uses GPT-5.4; best baseline is GPT-5.5 (43.5%). A same-model ablation is planned.
- **SWE-bench Lite:** Baseline estimated from public Copilot agent reports. No verified same-model-without-Squad run performed.
- **TerminalBench 2.0:** Same model (Claude Opus 4.6), controlled. 20 of 89 available tasks sampled, selected by sequential order (not cherry-picked). The two conditions ran through slightly different harnesses — Squad via a host-side orchestrator (`run-terminalbench-host-task.ps1`) and the baseline via the native terminal-bench harness — but both execute the identical task workloads and are scored by each task's own Docker verifier. Squad *can* run under the native harness (verified in smoke tests); the full 89-task suite is future work, not a capability blocker.

Metric note: Metrics differ by benchmark. MARBLE (ablation charts) reports *completion* = agent produced an output file within a 600s timeout; the database domain was additionally re-graded for *correctness* using MARBLE's official recall metric (see "Completion vs. correctness"). DevBench: pass@1 (test execution). SWE-bench: pass@1 (patch applies + tests pass). TerminalBench: verifier script passes. We use "completion" where the metric only checks that output appeared, and "resolution/pass" where output is verified correct.

Variance: MARBLE ablation completion σ ($n=10$ per cell, across 4 domains): Full Squad $\sigma=0\%$, Coord-only $\sigma=4.3\%$, No-Squad $\sigma=11.2\%$, Memory-only $\sigma=26.8\%$. (An earlier version reported No-Squad $\sigma=35.4\%$; that was inflated by a fabricated 0% database figure, now corrected to 100% completion after re-running all 10 tasks.) Full 400-task run: $397/400 = 99.25\%$ (3 tasks timed out without output). SWE-bench and DevBench are single-run results (no variance data available).

Statistical power: With $n=10$ per cell, a two-proportion z-test can only detect effects of roughly $\sim 25\text{pp}$ at $p<0.05$. After correcting the database data, the same-model *completion* lift (full Squad 100% vs no-Squad 85% = +15pp) and the database *correctness* differences ($\leq 20\text{pp}$ between conditions) fall **below** this threshold — i.e., they are within noise and should be read as directional, not statistically significant. We report them honestly rather than overclaiming; larger runs are needed to establish significance.

Timing: MARBLE and ablation: June 2026. TerminalBench 2.0: June 2026. SWE-bench Lite: May 2026. DevBench: June 2026. All benchmarks ran within a 6-week window.

Cost: All costs from actual Claude API billing (input + output tokens). Memory-only cost not recorded — estimated at $\sim \$1.50/\text{task}$ based on context size.

